

WinItalPascal - Database

Gestione Database SQL Server

Il modulo Database di **WinItalPascal** semplifica la gestione delle connessioni SQL Server e delle operazioni più comuni.

La libreria evita di dover riscrivere continuamente:

- apertura connessioni;
- gestione SqlCommand;
- SqlDataAdapter;
- gestione parametri SQL;
- caricamento DataTable;
- gestione errori.

⚠ Configurazione obbligatoria App.config

Prima di utilizzare le funzioni Database è **necessario** inserire nel file:

```
App.config
```

la connection string con il nome:

```
MiaConnessione
```

Il nome deve essere esattamente:

```
MiaConnessione
```

Esempio:

```
<connectionStrings>

<add name="MiaConnessione"
  connectionString="Data Source=SERVER;
  Initial Catalog=DBClienti;
  Integrated Security=True;
  TrustServerCertificate=True"
  providerName="System.Data.SqlClient"/>

</connectionStrings>
```

Se la connection string non viene trovata verrà generato un errore:

```
Connection string 'MiaConnessione' non trovata in App.config
```

Classi disponibili

La gestione Database comprende:

```
DB
DBApri
DataGVLoad
```

Classe DB

La classe principale per l'accesso al database.

Permette di eseguire:

- SELECT;
 - INSERT;
 - UPDATE;
 - DELETE;
 - caricamento dati per DataGridView;
 - query parametrizzate.
-

Apertura connessione

Esempio:

```
Using conn = DB.GetConnection()  
  
    conn.Open()  
  
    ' operazioni database  
  
End Using
```

La libreria recupera automaticamente:

```
MiaConnessione
```

dal file App.config.

ExecuteScalar

Utilizzato per ottenere un singolo valore.

Esempi:

- COUNT;
- MAX;
- MIN;
- ID appena inserito.

Esempio:

```
Dim totale As Integer =  
    Convert.ToInt32(  
        DB.ExecuteScalar(  
            "SELECT COUNT(*) FROM Clienti")  
    )
```

ExecuteNonQuery

Utilizzato per:

- INSERT;
- UPDATE;
- DELETE.

Esempio:

```
Dim risultato As Integer =  
    DB.ExecuteNonQuery(  
        "DELETE FROM Clienti WHERE IdClienti=10")
```

Il valore restituito indica il numero di righe modificate.

FillDataTable

Carica i dati in un DataTable.

Utilizzato principalmente per:

- DataGridView;
- elaborazioni dati;
- esportazioni.

Esempio:

```
Dim dt As DataTable =  
    DB.FillDataTable(  
        "SELECT * FROM Clienti")  
  
DataGridView1.DataSource = dt
```

FillDataSet

Permette di caricare più tabelle.

Esempio:

```
Dim ds As DataSet =  
    DB.FillDataSet(  
        "SELECT * FROM Clienti")
```

Query semplificata

Permette di creare query parametrizzate senza scrivere manualmente i parametri.

Esempio:

```
Dim dt As DataTable =  
    DB.Query(  
        "SELECT * FROM Clienti WHERE Citta LIKE @p1",  
        "%" & TxtCitta.Text & "%")
```

Query con più parametri

Esempio:

```
Dim dt As DataTable =  
    DB.Query(  
        "SELECT * FROM Clienti  
        WHERE Citta LIKE @p1  
        AND CAP LIKE @p2",  
        "%" & TxtCitta.Text & "%",  
        "%" & TxtCAP.Text & "%")
```

QueryLike

Versione semplificata per ricerche LIKE.

Esempio:

```
Dim dt As DataTable =  
    DB.QueryLike(  
        "SELECT * FROM Clienti WHERE Citta LIKE @p1",  
        TxtCitta.Text)
```

```
"SELECT * FROM Clienti WHERE Cliente LIKE @p1",  
TxtCliente.Text)
```

La libreria aggiunge automaticamente:

```
%
```

prima e dopo il testo cercato.

Recupero elenco Tabelle

Esempio:

```
Dim dt As DataTable =  
    DB.GetTables()
```

Restituisce tutte le tabelle presenti nel database.

Classe DBApri

Classe alternativa per operazioni database più semplici.

Funzioni disponibili:

```
GetConnection()  
GetDataTable()  
ExecuteScalar()  
ExecuteNonQuery()
```

Esempio GetDataTable

```
Dim dt As DataTable =  
    DBApri.GetDataTable(  
        "SELECT * FROM Clienti")  
  
DataGridView1.DataSource = dt
```

Classe DataGVLoad

Classe dedicata al caricamento diretto dei DataGridView.

Permette di collegare una query SQL direttamente alla griglia.

Esempio:

```
DataGVLoad.ApriDGV(  
    DgvClienti,  
    "SELECT * FROM Clienti")
```

La libreria:

- apre la connessione;
 - esegue la query;
 - carica il DataTable;
 - assegna il DataSource.
-

Query con parametri

Esempio:

```
Dim parametri As New List(Of SqlParameter)  
  
parametri.Add(  
    New SqlParameter("@Cliente",  
        TxtCliente.Text))  
  
DataGVLoad.ApriDGV(  
    DgvClienti,  
    "SELECT * FROM Clienti  
    WHERE Cliente=@Cliente",  
    parametri)
```

Gestione errori

Le classi Database integrano il sistema:

```
FrameworkLogger
```

Gli errori vengono registrati nel file:

```
ItalPascal_Log.txt
```

presente nella cartella dell'applicazione.

Esempio completo

Caricamento Clienti:

```
Private Sub FrmClienti_Load(  
sender As Object,  
e As EventArgs) Handles MyBase.Load  
  
    Try  
  
        DataGVLoad.ApridGV(  
            DgvClienti,  
            "SELECT * FROM Clienti")  
  
    Catch ex As Exception  
  
        FrameworkLogger.LogError(  
            ex,  
            "Caricamento Clienti")  
  
    End Try  
  
End Sub
```

Note importanti

La libreria utilizza:

`System.Data.SqlClient`

Compatibile con:

- SQL Server;
- SQL Server Express;
- LocalDB.

Documentazione correlata

`README.md`
`README_GridUtility.md`
`README_Reports.md`
`README_Forms.md`
`README_Logging.md`
`README_Popup.md`
`README_IPMessageBox.md`

Video Tutorial

Video dedicato al modulo Database:

[video Database](#)

Repository GitHub

[Repository GitHub](#)

Pacchetto NuGet

`WinItalPascal`

WinItalPascal

Utility Library for VB.NET WinForms

Versione documentazione: 2.0.6

